

Trabalho 4 - Sistema de Autocomplete

Estrutura de Dados

Alunos: Joel Gil Medeiros de Oliveira | Osmar Caio da Costa Silva

1 Representação da Trie

A estrutura principal utilizada no trabalho foi uma *Trie*, ou árvore de prefixos. Nessa estrutura, cada caminho a partir da raiz representa uma sequência de caracteres. Quando esse caminho corresponde ao título completo de um jogo, o último nó é marcado como fim de título e armazena um ponteiro para o respectivo objeto da classe `Game`.

Cada nó possui um array de ponteiros para filhos, uma variável booleana indicando se aquele nó representa o fim de um título e um ponteiro para `Game`. A Trie não cria novas cópias dos jogos; ela apenas guarda ponteiros para os objetos já existentes no array definido em `GamesDatabase.cpp`. Isso evita duplicação desnecessária dos dados principais do catálogo.

O alfabeto utilizado possui tamanho 36, correspondendo aos 10 dígitos e às 26 letras minúsculas. Os caracteres de '0' a '9' são mapeados para as posições de 0 a 9 do array de filhos. As letras de 'a' a 'z' são mapeadas para as posições de 10 a 35. Assim, cada caractere válido da chave de busca pode ser convertido diretamente para uma posição do array.

Antes de inserir ou buscar um título, o texto é convertido pelo método `toSearchKey`. Essa conversão transforma letras maiúsculas em minúsculas, remove espaços em branco e preserva letras e números. Por exemplo, `Half Life`, `HALF LIFE` e `halflife` geram a mesma chave: `halflife`. Com isso, a busca se torna *case-insensitive* e também ignora espaços.

2 Funcionamento dos Métodos

O método `insert` recebe um ponteiro para um objeto `Game`. Primeiro, o título do jogo é convertido para a chave de busca. Em seguida, a Trie é percorrida caractere por caractere. Quando o filho correspondente a um caractere ainda não existe, um novo nó é criado. Ao final do caminho, o último nó é marcado como fim de título e passa a armazenar o ponteiro para o jogo inserido.

O método `contains` verifica se existe um jogo com o título informado. Para isso, o texto recebido também é convertido para chave de busca. Depois, a Trie é percorrida seguindo os caracteres dessa chave. Se algum ponteiro necessário não existir, o método retorna `false`. Se todo o caminho for percorrido, o método retorna `true` apenas se o último nó estiver marcado como fim de título.

O método `autocomplete` recebe um prefixo e um valor `k`. Caso `k <= 0`, o método retorna um vetor vazio. Caso contrário, o prefixo é convertido para chave de busca e a

Trie é percorrida até o nó correspondente a esse prefixo. Se esse caminho não existir, nenhum resultado é encontrado. Se existir, a subárvore abaixo desse nó é percorrida para coletar todos os jogos compatíveis com o prefixo.

Após a coleta, os resultados são ordenados pelo método `sortResults`, sem uso de `std::sort`. O critério principal é a maior popularidade. Em caso de empate, vem primeiro o jogo cuja chave de busca do título aparece antes em ordem alfabética. Por fim, o método retorna no máximo os primeiros k jogos do vetor ordenado.

3 Análise de Custo

Seja L o tamanho do título após a conversão para chave de busca, p o tamanho do prefixo convertido, s o número de nós visitados na subárvore do prefixo e r o número de jogos encontrados.

A inserção custa $O(L)$, pois percorre uma vez os caracteres do título convertido e cria, no máximo, um nó por caractere. A busca exata com `contains` também custa $O(L)$, já que segue apenas o caminho do título na Trie.

No autocomplete, primeiro há custo $O(p)$ para chegar ao nó correspondente ao prefixo. Depois, a subárvore desse nó é percorrida para coletar os jogos encontrados, com custo $O(s)$. Em seguida, os r resultados são ordenados manualmente. Como a ordenação compara pares de elementos, o custo no pior caso é $O(r^2)$. Considerando que a comparação em caso de empate pode envolver a chave do título, o custo pode ser descrito como $O(r^2L)$ no pior caso.

4 Comparação com Solução Ingênua e Uso de Memória

Uma solução ingênua para o autocomplete seria percorrer todo o array de jogos a cada consulta, converter cada título para chave de busca e verificar se ele começa com o prefixo informado. Se houver n jogos, essa etapa teria custo $O(nL)$ apenas para encontrar candidatos, além do custo de ordenação dos resultados.

Com a Trie, a busca não precisa percorrer diretamente toda a base. O algoritmo segue o caminho do prefixo em $O(p)$ e depois explora apenas a subárvore correspondente aos títulos compatíveis. Quando o prefixo é seletivo, essa estratégia visita muito menos nós do que uma varredura completa do catálogo. No pior caso, se o prefixo for vazio ou muito genérico, a subárvore pode se aproximar da Trie inteira, mas o método ainda respeita a proposta de busca por prefixos usando a estrutura principal.

Em relação à memória, a Trie consome mais espaço do que um array simples de jogos. Cada nó armazena 36 ponteiros para filhos, uma indicação de fim de título e um ponteiro para `Game`. Se N é o número de nós, o espaço usado pela Trie é proporcional a $O(36N)$, isto é, $O(N)$ em notação assintótica.

Apesar desse custo adicional, há compartilhamento de prefixos. Títulos que começam com a mesma sequência de caracteres reutilizam os mesmos nós iniciais, reduzindo a quan-

tidade total de nós em comparação com armazenar cada chave de forma completamente independente. O tamanho do alfabeto influencia diretamente a memória: um alfabeto maior aumenta o array de filhos em cada nó, enquanto um alfabeto fixo com letras e dígitos torna o acesso simples e direto.